

Informe acerca del simulador de parqueadero

“PARQUEADERO LAS MARGARITAS”

1. Descripción del sistema y analogía con SO

Este simulador de parqueaderos contiene las siguientes partes principales:

El semáforo (semaforo = threading.Semaphore(capacidad)) es el encargado de controlar y coordinar el acceso a los parqueaderos (espacios), ya que este es un recurso compartido del sistema. Varios procesos (vehículos) podrían ingresar al mismo tiempo a un recurso (parqueaderos), generando condiciones de carrera y arrojando resultados erróneos sobre la ocupación del parqueadero. En un sistema operativo podría generar inconsistencias en la administración de recursos, como contadores incorrectos de memoria disponible, acceso simultáneo a dispositivos o uso incorrecto de recursos compartidos.

Los locks asignados a las variables estados y ocupados protegen dos recursos compartidos del sistema. Una analogía en un SO sería varios programas accediendo al mismo recurso al mismo tiempo, por ejemplo, un editor de texto como Word guardando un archivo mientras el sistema operativo actualiza la hora de modificación del mismo archivo. Para evitar conflictos, los *locks* permiten que solo una sección crítica se ejecute a la vez, impidiendo que dos procesos escriban simultáneamente en el diccionario de estados y asegurando que el contador de ocupados ejecute `ocupados += 1` para un solo proceso a la vez.

La función principal inicio_simulacion() representa el *boot* del sistema. Empieza una sola vez al dar clic en el botón “Iniciar simulación”, prepara el sistema creando los hilos, indica cuándo deben iniciar y espera a que todos terminen para finalizar la simulación. Allí también se crea el archivo principal donde se guardan los logs de eventos. Cada cambio de estado genera un evento, registrando estado, contador de ocupados y tiempo. En la analogía con un SO, esto se asemeja a cuando el sistema arranca y el *kernel* ya se encuentra cargado y administrando los procesos activos.

El botón “**Reiniciar simulación**”, conectado con la función `reiniciar()`, representa el *reboot* del sistema. Funciona de forma similar a cuando reiniciamos una computadora: se detienen los procesos activos, se limpian los logs (registros) y todas las variables se vuelven a inicializar, dejando un “lienzo en blanco” para los nuevos procesos que vayan a entrar.

El botón “**Pausar/Reanudar**” llama a la función `pausarReanudar()`. Esta verifica si la simulación (procesos) está en ejecución. Si `simu_evento` está en estado `set` (activo),

la simulación se detiene; si está en estado *clear*, la simulación se reanuda. La función **pausa_inteligente(segundos)** hace que los hilos (procesos) esperen por un tiempo determinado. Durante ese tiempo quedan bloqueados en la línea **simu_evento.wait()**, revisando periódicamente el estado del evento cada 0.1 segundos hasta que vuelva a estar en set para continuar su ejecución.

El historial de logs representa el historial del sistema. No interviene con ningún proceso; únicamente muestra lo que el simulador ejecutó. En este simulador, los registros se guardan en un archivo CSV, donde cada fila representa un evento que ocurre cada vez que un hilo (vehículo) cambia de estado. En la interfaz se habilitaron únicamente los estados NUEVO, ESPERANDO, ESTACIONADO y SALIENDO; el estado INACTIVO no se muestra en la interfaz, pero sí queda registrado en los logs. Estos estados hacen analogía con estados clásicos de procesos en un sistema operativo, como new, blocked/waiting, running y terminated

2. Explicación del uso de hilos y sincronización

Se utilizan hilos (*threads*) para simular varios vehículos que ingresan, quedan estacionados y salen del parqueadero de forma concurrente. Cada vehículo se ejecuta como un hilo independiente mediante la siguiente línea de código:

```
carrito = threading.Thread(target=vehiculo,args=(i,))
```

Esto permite representar un sistema operativo real, donde varios procesos se ejecutan concurrentemente.

Para evitar errores de concurrencia se utilizan los siguientes mecanismos de sincronización:

Semáforo (Semaphore): controla la cantidad de vehículos que pueden ingresar y permanecer estacionados en el parqueadero. De esta manera, se asegura que **nunca se supere la capacidad máxima del recurso disponible**.

Locks (Lock): protegen **variables críticas** como el contador de ocupados y los estados de los vehículos. Así se evitan inconsistencias en los datos por acceso simultáneo de varios hilos.

Evento (Event): se utiliza para controlar el comportamiento general de los hilos.

Funciona como el interruptor global del sistema, permitiendo pausar y reanudar la simulación de manera controlada.

```
simu_evento = threading.Event()
```

3. captura de pantalla del simulador en ejecución:

